Ubuntu Server OS

Production System Architecture & Security Specification

ROLE	TARGET OS	STATUS	DATE
Senior Software Architect	Ubuntu Server 22.04 / 24.04 LTS	Pending Approval	October 2026

1. System Overview

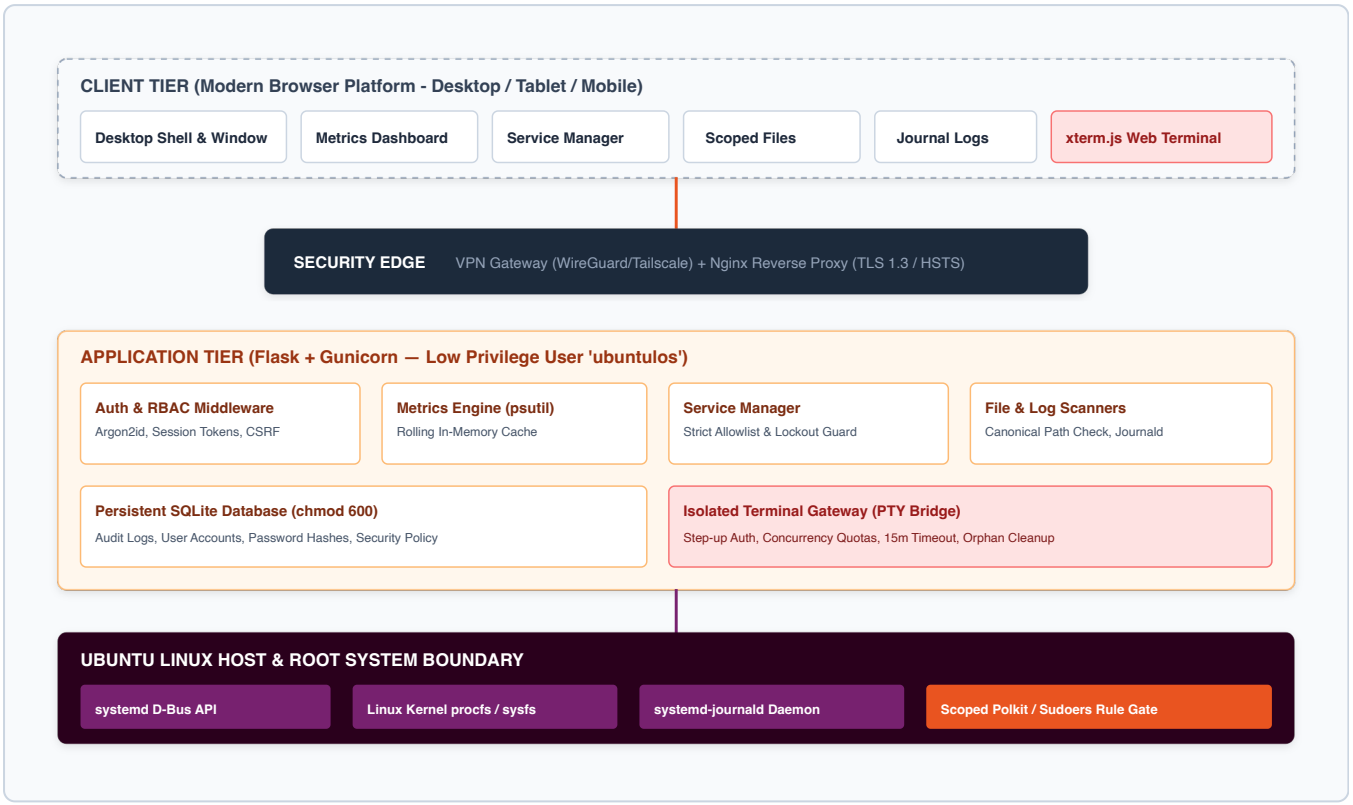
Ubuntu Server OS is an enterprise-grade, browser-based administrative control environment for Ubuntu Server installations. Accessible across desktop, tablet, and mobile browsers, it delivers system telemetry, systemd service management, restricted file access, journald log observation, and an isolated browser terminal gateway without requiring continuous direct SSH console access.

Core Architectural Flow

Browser requests reach the underlying operating system through four strictly isolated tiers:

1. **Client Tier (Browser)**: Built with standard HTML5, CSS custom properties, and native ES modules. Issues authenticated REST requests for management operations and uses WebSockets for telemetry feeds and terminal sessions.
2. **Perimeter Tier (Reverse Proxy & TLS)**: Nginx or Caddy terminates TLS 1.3, enforces HSTS, applies rate limits, and proxies requests over a protected local Unix domain socket (`/run/ubuntu-server-os/web.sock`).
3. **Application Tier (Flask + Gunicorn)**: Runs under a dedicated, low-privilege system account (`ubuntulos` with `/usr/sbin/nologin`). Enforces session validation, CSRF checks, RBAC authorization, and input validation. Direct shell access and root execution are strictly prohibited.
4. **Privileged Execution Tier (System Boundary)**: Administrative changes (service restarts, host reboot) interface with `systemd` via the Linux D-Bus API (`org.freedesktop.systemd1`) or a tightly scoped privileged helper script managed under `/etc/sudoers.d/` with strict parameter validation.

2. Architecture Diagram



3. Recommended Project Structure

```
ubuntu-server-os/
├── pyproject.toml                  # Project metadata, dependencies, linting, pytest configs
├── gunicorn.conf.py                # Production WSGI config (Unix socket, worker lifecycle)
├── deploy/                         # System deployment & operational templates
│   ├── nginx/ubuntu-server-os.conf  # Nginx reverse proxy, TLS 1.3, rate-limiting & CSP
│   ├── systemd/ubuntu-os.service    # Systemd service unit for Flask backend daemon
│   └── sudoers.d/99-ubuntu-os       # Tightly scoped sudoers permissions for helper script
├── server/                         # Python Flask Backend
│   ├── __init__.py                 # Application Factory ()
│   ├── config.py                   # Environment profiles (Development, Production, Testing)
│   ├── extensions.py               # Database, Session, and CSRF extensions
│   ├── api/v1/                     # REST API Controllers (auth, metrics, services, fs, logs)
│   ├── core/                       # Hardware & System Interfacing Layer
│   │   ├── auth_manager.py         # Password hashing (argon2id), session lifecycle
│   │   ├── metrics_collector.py    # psutil sampler with ring-buffer cache
│   │   ├── service_manager.py      # D-Bus systemd interface & allowlist enforcer
│   │   ├── file_manager.py         # Realpath canonicalizer & path-jail guards
│   │   ├── log_reader.py           # Journald integration & filtered streams
│   │   └── audit_logger.py         # Persistent tamper-evident audit trail
│   ├── terminal/                   # Isolated Terminal Gateway (WebSocket & PTY manager)
│   └── models/                     # SQLite Schema (User, AuditLog, AllowedService)
├── client/                         # Browser Frontend (Vanilla HTML5, CSS3, ES Modules)
│   ├── index.html                  # Desktop OS shell
│   ├── css/                        # Design tokens, layout, window frames, dock, responsive
│   └── js/                         # App bootstrap, reactive state, event bus, app panels
└── tests/                          # Multi-tier Automated Test Suite
    ├── unit/                       # Parsers, path validators, regex checks
    ├── integration/                # API contracts, auth, mocked systemd
    ├── security/                   # Path traversal, command injection, brute-force batteries
    └── frontend/                   # Viewport responsive layout & smoke tests
```

4. Frontend Architecture

Desktop Shell & Window Manager

- **Windowing Engine**: Movable, resizable, minimizable, and stackable window containers with z-index elevation.
- **Dock & Top Bar**: Ubuntu-themed launcher dock with running indicators; system top bar with host telemetry sparklines.
- **Adaptive Responsive Engine**: Viewports < 768px (smartphones) automatically switch from floating windows to full-screen card views with touch-friendly back bars.

Design Tokens & Vanilla ESM

- **Color Palette**: Canonical Aubergine (`#77216F`), Ubuntu Orange (`#E95420`), Slate Surface (`#2D2D2D`), Jet Black (`#111111`).
- **Native ES Modules**: Zero bundler overhead. Direct browser imports eliminate node build steps on the production server.
- **State & Event Bus**: Reactive store using JS `Proxy` paired with an event bus for inter-window communications.

5. Backend Architecture

- **Application Factory Pattern**: Standardized Flask factory instantiating database bindings, CSRF verification, and security headers.
- **Sub-Millisecond Cached Metrics**: A dedicated background thread samples `psutil` every 2 seconds into a memory ring-buffer. Client API queries read directly from cache without triggering disk or procfs CPU spikes.
- **Consistent JSON Envelopes**: Standardized responses (`status`, `data`, `message`, `code`) with suppressed error tracebacks in production.
- **Audit Logging**: Every state-modifying action (service restart, file modification, login attempt) writes an immutable record to SQLite.

6. Proposed API Contract

Endpoint	Method	Purpose	Authorization
`/api/v1/auth/login`	`POST`	Authenticate credentials, issue session cookie & CSRF token	PUBLIC (RATE-LIMITED)
`/api/v1/auth/logout`	`POST`	Invalidate session token and terminate session	AUTHENTICATED
`/api/v1/metrics/current`	`GET`	Instantaneous CPU, memory, disk, and network stats	AUTHENTICATED
`/api/v1/metrics/history`	`GET`	Rolling 60-point time-series metrics for mini charts	AUTHENTICATED
`/api/v1/services`	`GET`	List allowlisted systemd services with active status	AUTHENTICATED
`/api/v1/services/<name>/action`	`POST`	Execute `start`, `stop`, `restart`, or `reload`	ADMIN ONLY
`/api/v1/fs/browse`	`GET`	Directory listing inside allowlisted root paths	AUTHENTICATED
`/api/v1/fs/read`	`GET`	View text content of allowlisted file	AUTHENTICATED
`/api/v1/fs/write`	`PUT`	Save changes to allowlisted file	ADMIN ONLY
`/api/v1/logs/journal`	`GET`	Paginated query of journald entries by unit/priority	AUTHENTICATED
`/api/v1/terminal/session`	`POST`	Request single-use token for PTY WebSocket bridge	ADMIN (STEP-UP AUTH)
`/api/v1/audit/events`	`GET`	View administrative action audit log	ADMIN ONLY

7. Security Architecture

> **Mandatory Security Guarantees**
>
> The Flask application **never runs as root**. It executes under user `ubuntulos`. No generic shell command execution endpoint is exposed. System modifications occur through validated D-Bus calls or explicitly allowlisted helper arguments.

Lockout & Denial-of-Service Defense

- **Critical Service Blacklist**: Services such as `ssh`, `systemd-networkd`, `NetworkManager`, and VPN daemons (`wireguard`, `tailscaled`) are hard-locked against stop/disable commands from the web UI.
- **Brute-Force Lockout**: Accounts automatically lock for 15 minutes after 5 consecutive failed login attempts; IP rate limits are enforced at the Nginx edge.

Path Traversal & Injection Neutralization

- **Canonical Path Validation**: Every filesystem request uses `os.path.realpath()` to resolve symlinks and verify paths remain strictly within designated directory jails.
- **Zero Shell Interpolation**: All OS commands use direct argument vectors: `subprocess.run([binary, arg1, arg2], shell=False)`.

8. Terminal Subsystem Design

The browser terminal is isolated from the primary REST API to protect worker threads from starvation and minimize security exposure:

- **Separation from SSH**: Native Ubuntu SSH uses public key authentication directly into PAM login shells. The browser terminal runs an isolated PTY worker process over authenticated WebSockets.
- **Mandatory Step-Up Re-Authentication**: Administrators must re-enter their credentials to generate a single-use, 60-second WebSocket ticket.
- **Hard Quotas & Cleanup**: Maximum 2 concurrent terminal sessions per administrator, automatic disconnection after 15 minutes of idle time, and mandatory `SIGHUP` and `SIGKILL` signal cascading on WebSocket disconnect to eliminate orphaned shell processes.

9. Testing Strategy

- **Unit Tests**: Parameter validators, regex allowlists, path canonicalization, and psutil telemetry parsers.
- **Security Test Battery**: 50+ directory traversal vectors (`../../etc/shadow`, encoded slashes, symlinks), command injection attacks, and session tampering.
- **Integration Tests**: Mocked systemd D-Bus service state transitions, CSRF token verification, and session lifecycle.
- **End-to-End & Responsive Tests**: Automated headless browser verification across desktop (1920x1080), tablet (768x1024), and mobile (375x812) viewports.

10. Implementation Roadmap

Phase	Scope & Deliverables	Key Acceptance Criteria
Phase 1	Core Foundation, App Factory, SQLite Models, Argon2id Auth & CSRF	Session tokens issued; 5 failed logins lock account; audit trail active.
Phase 2	OS-Style Frontend Shell, Top Bar, Dock, Window Manager & Metrics Panel	Responsive UI across mobile/desktop; CPU/RAM/Disk/Network polls smoothly.
Phase 3	Allowlisted Systemd Service Manager & Lockout Safeguards	Only allowed services listed; SSH/Network protected from accidental stop.
Phase 4	Scoped File Manager & Journald Log Viewer	Path traversal 100% blocked; sensitive system files masked; logs searchable.
Phase 5	Browser Terminal Gateway (xterm.js + PTY Bridge)	Step-up auth required; idle timeouts enforce clean process termination.
Phase 6	Production Hardening, Nginx Config, Systemd Units & Final Verification	Automated deployment verified on clean Ubuntu 22.04/24.04 LTS instance.

11. Technical Decisions & Open Questions

The following architectural decisions are presented for your explicit review and confirmation:

1. **Authentication Mode**: *Standalone SQLite user accounts with Argon2id* (Recommended to keep web credentials decoupled from OS accounts) vs. *Direct PAM Linux user auth*.
2. **File Manager Permitted Roots**: Recommendation to jail file operations to `/var/www`, `/var/log`, and `/home`.
3. **Terminal Rollout**: Recommendation to implement the browser terminal in **Phase 5** after validating Phases 1–4.

12. Architecture Review Summary

> **Recommended First Implementation Step:** Phase 1 — Core Foundation & Security Shell. This establishes the application factory, database schemas, Argon2id credential validation, CSRF middleware, and automated test harnesses before introducing UI components or system operations.